S1. Tarea. Exploradores NoSQL

Ajuste y ampliación del modelo documental de Hospitalización — Base de Datos 2, Unidad 3

Estudiante: Anderson Cañas Alcaraz

Programa: Tecnología en Desarrollo de Software — Semestre 4

Motor de base de datos: MongoDB (documental), base `hce_hospitalizacion_grupo_6`

Contexto narrativo: Sistema de hospitalización — Secretaría de Salud de Antioquia

Modalidad de trabajo: Individual (sin equipo asignado)

1. Introducción y continuidad con la Unidad 2

Este trabajo continúa la actividad de la Unidad 2 ("Construyendo una base de datos NoSQL"), donde se diseñó e implementó el subsistema de Hospitalización de una Historia Clínica Electrónica usando MongoDB, con las colecciones `pacientes`, `hospitalizaciones` (que embebe `diagnosticos` y `alta`), `notas_evolucion`, `signos_vitales`, `ordenes_medicamento` y `notas_enfermeria`.

La Unidad 3 pide ampliar ese modelo cubriendo explícitamente dos entidades que antes no existían como tales: **médicos** y **camas**. No se creó una base nueva ni se borró nada de lo anterior: se trabajó sobre la misma base `hce_hospitalizacion_grupo_6`, agregando lo que faltaba.

> **Nota sobre el trabajo en equipo:** la nomenclatura del curso sigue hablando de "equipos", pero esta actividad, igual que la anterior, se desarrolló de forma individual porque no se asignaron compañeros de grupo. Ver `division_trabajo.md`.

1.1 Tecnología elegida

Se mantiene **MongoDB** (modelo documental) en vez de un modelo de grafos (Neo4j), por la misma razón que en la Unidad 2: la información de una hospitalización es naturalmente jerárquica (un paciente tiene hospitalizaciones, una hospitalización tiene diagnósticos, notas, signos vitales y órdenes), y ese tipo de relación "contiene / pertenece a" se modela de forma muy natural con documentos y referencias, sin necesitar la flexibilidad de un grafo para relaciones muchos-a-muchos complejas.

2. Diseño del modelo: qué se ajustó

2.1 Qué faltaba y qué se agregó

Entidad	Unidad 2	Unidad 3 (este trabajo)
Médicos	Solo existía como texto suelto (ej. `medico_tratante: "Dr. Andres Felipe Suarez"`) dentro de otras colecciones.	Colección propia `medicos`, referenciada desde `hospitalizaciones`, `ordenes_medicamento` y `notas_evolucion` mediante `medico_id`; incluye un contador denormalizado `total_altas_dadas`.
Camas	No existía ningún concepto de cama; solo el campo `servicio` (texto).	Colección propia `camas`, referenciada desde `hospitalizaciones` mediante `cama_id`, con estado `libre` / `ocupada` / `mantenimiento`, piso y servicio.

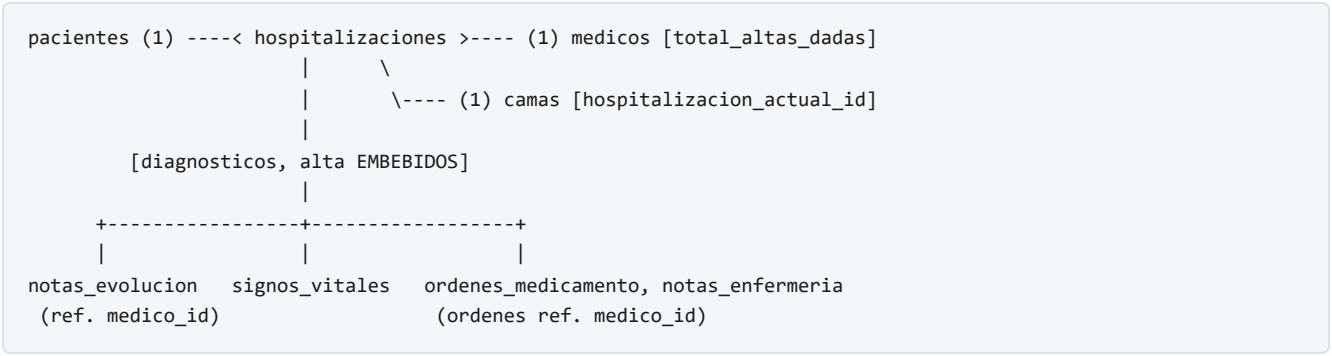
Los campos de texto originales (`medico_tratante`, `medico`, `medico_ordena`) **no se eliminaron**: se conservan como una "foto" histórica del nombre del médico en el momento del registro, mientras que el nuevo campo `*_id` se usa para las consultas en vivo. Esto evita romper nada de lo ya entregado en la Unidad 2.

2.2 Justificación: embeber vs. referenciar

Colección / campo	Estrategia	Por qué
`medicos`	Referenciada (colección propia)	Un mismo médico aparece en decenas o cientos de hospitalizaciones, órdenes y notas a lo largo del tiempo. Embeberlo duplicaría su nombre, especialidad y registro profesional en cada uno de esos documentos.
`camas`	Referenciada (colección propia)	Una cama se reutiliza con pacientes distintos constantemente, y el hospital necesita poder preguntar "¿cuántas camas libres hay en Medicina Interna ahora mismo?" de forma instantánea.
`camas.hospitalizacion_actual_id`	Denormalización controlada	Se podría calcular buscando en `hospitalizaciones`, pero esa búsqueda se haría todo el tiempo. Guardar una copia controlada hace la consulta instantánea, a cambio de que la aplicación la mantenga sincronizada en cada ingreso/alta.
`medicos.total_altas_dadas`	Denormalización controlada (contador)	Igual lógica: se podría calcular siempre con un `$lookup` (como hace la Consulta 2), pero un contador ya sumado sirve para un reporte rápido tipo "ranking de médicos". El costo es que la aplicación debe sumarlo con `$inc` en el momento correcto (ver sección 3.4 y la reflexión ética sobre trazabilidad).

El esquema completo, con el detalle de cada colección y cada decisión, está documentado en `schema.json`.

2.3 Diagrama conceptual (texto)


pacientes (1) ----< hospitalizaciones >---- (1) medicos [total_altas_dadas]
(1) camas [hospitalizacion_actual_id]
[diagnosticos, alta EMBEBIDOS]
notas_evolucion
(ref. medico_id)
signos_vitales
ordenes_medicamento, notas_enfermeria
(ordenes ref. medico_id)

3. Implementación

3.1 Creación de las colecciones nuevas

Script: `01_create_medicos_camas.js`. Crea `medicos` y `camas` con validación `$jsonSchema` e índices (`registro_medico` único en médicos, `numero_cama` único en camas), y actualiza con `collMod` el validador de `hospitalizaciones`, `ordenes_medicamento` y `notas_evolucion` para documentar oficialmente el nuevo campo `medico_id` / `cama_id`.

> Evidencia real de esta ejecución: ver Figura 1, sección 5.1.

3.2 Carga de datos simulados

Script: `02_insert_medicos_camas.js`. Carga 7 médicos (los 5 que ya existían como texto en la Unidad 2, más 2 nuevos sin pacientes asignados todavía, todos con `total_altas_dadas: 0` al inicio) y 16 camas repartidas en 4 servicios (UCI, Cirugía General, Medicina Interna, Ortopedia y Traumatología).

> Evidencia real de esta ejecución: ver Figura 2, sección 5.1.

3.3 Vinculación de los datos existentes

Script: `03_vincular_existentes.js`. Conecta las 10 hospitalizaciones que ya existían con su médico real (por coincidencia de nombre) y con una cama real del mismo servicio, actualiza el estado de esas camas, y recalcula el contador `total_altas_dadas` de cada médico contando sus altas históricas.

> Evidencia real de esta ejecución: ver Figuras 3 y 4, sección 5.1.

3.4 CRUD completo

Script: `04_crud.js`. Todas las operaciones se ejecutaron contra la base real y fueron verificadas. Cada bloque del script incluye un comentario explicando **por qué** se eligió esa forma de hacerlo (qué operador de MQL y por qué, no solo qué hace).

Operación	Qué hace y por qué
CREATE	Inserta un paciente, su hospitalización (ya con `medico_id` y `cama_id`) y una orden de medicamento. El insert de la hospitalización actualiza también la cama en el mismo paso, porque "paciente en una cama" es un solo hecho visto desde dos colecciones.
UPDATE simple	Cambia el teléfono de un paciente con `$set` directo — campo de primer nivel, sin anidamiento.
UPDATE anidado (objeto)	Cambia `contacto_emergencia.telefono` con notación de punto, en vez de reemplazar todo el subdocumento (que borraría `nombre` y `parentesco` si no se incluyen).

UPDATE anidado (arreglo)	Agrega un diagnóstico secundario con `$push`, no con `$set` del arreglo completo, para no borrar el diagnóstico principal que ya existía.
MULTI-COLECCIÓN	**Dar de alta a un paciente.** Un solo hecho de negocio que toca 3 colecciones relacionadas: (1) `hospitalizaciones` — se llena por primera vez el subdocumento `alta` y cambia `estado`; (2) `camas` — la cama que ocupaba vuelve a `"libre"`; (3) `medicos` — se suma 1 a `total_altas_dadas` con `$inc`. Se hace en 3 escrituras separadas porque esta instalación de MongoDB no usa transacciones multi-documento; si el proceso se interrumpiera entre escrituras, las 3 colecciones quedarían inconsistentes entre sí — riesgo real que se retoma en la reflexión ética (sección 7).
SOFT DELETE	Marca a un paciente como `activo: false` en vez de borrarlo, porque la ley exige conservar la historia clínica (continuidad de la atención y trazabilidad legal).
HARD DELETE	Inserta y `deleteOne` real de un registro de prueba, creado solo para esta demostración — aquí sí es correcto el borrado definitivo porque no es un dato clínico real ni tiene obligación legal de conservarse.

Evidencia real de esta ejecución (las 7 operaciones completas): ver Figuras 5 a 8, sección 5.1.

4. Consultas de agregación avanzadas

Las 4 consultas siguientes son nuevas frente a las ya entregadas en la Unidad 2 y en el Foro U3: aquellas se enfocaban en diagnósticos puntuales, evolución cronológica, signos vitales y conteo de pacientes por servicio. Estas 4 aprovechan específicamente las colecciones nuevas (`medicos`, `camas`) para detectar patrones operativos y clínicos distintos, y cada una explica además por qué le importaría a la Secretaría de Salud de Antioquia. Script completo: `05_consultas_avanzadas.js`.

4.1 Consulta 1 — Ocupación de camas por servicio

Pregunta que responde: ¿cuántas camas libres y ocupadas tiene cada servicio en este momento, y qué porcentaje de ocupación representa?

> **Relevancia:** con hospitales públicos reportando esto, la Secretaría de Salud podría ver en un tablero en qué servicios ya no hay camas disponibles y decidir a dónde remitir pacientes nuevos, en vez de enterarse del problema cuando ya no hay dónde atender a alguien.

Etapas: `$match` → `$group` → `$project` → `$sort`.

```
db.camas.aggregate([
  { $match: { estado: { $in: ["libre", "ocupada"] } } },
  { $group: { _id: "$servicio", total_camas: { $sum: 1 },
      ocupadas: { $sum: { $cond: [{ $eq: ["$estado", "ocupada"] }, 1, 0] } } } },
  { $project: { _id: 0, servicio: "$_id", total_camas: 1, ocupadas: 1,
      libres: { $subtract: ["$total_camas", "$ocupadas"] },
      porcentaje_ocupacion: { $round: [{ $multiply: [{ $divide: ["$ocupadas", "$total_camas"] }, 100] }, 1] } }
},
  { $sort: { porcentaje_ocupacion: -1 } }
])
```

> Evidencia real de esta ejecución: ver Figura 9, sección 5.2.

4.2 Consulta 2 — Carga de hospitalizaciones por médico

Pregunta que responde: ¿qué médico tiene más hospitalizaciones a cargo (en total y activas en este momento), incluyendo a los médicos que todavía no tienen ningún paciente asignado?

> **Relevancia:** permite ver si la carga de trabajo está repartida de forma pareja entre el personal médico de un servicio, clave para decidir dónde reforzar turnos. Esta consulta calcula el total EN VIVO con `$lookup` (siempre exacto); el contador `medicos.total_altas_dadas` es la alternativa denormalizada para un reporte aún más rápido — ambas formas se documentan y se comparan en el script.

Etapas: `$lookup` → `$project` → `$sort`.

```
db.medicos.aggregate([
  { $lookup: { from: "hospitalizaciones", localField: "_id",
      foreignField: "medico_id", as: "hospitalizaciones_atendidas" } },
  { $project: { _id: 0, medico: "$nombre_completo", especialidad: 1,
```

```
      total_hospitalizaciones: { $size: "$hospitalizaciones_atendidas" },
      activas_ahora: { $size: { $filter: { input: "$hospitalizaciones_atendidas",
          as: "h", cond: { $eq: ["$$h.estado", "activa"] } } } } } },
  { $sort: { total_hospitalizaciones: -1 } }
])
```

Evidencia real de esta ejecución: ver Figuras 9 y 10, sección 5.2.

4.3 Consulta 3 — Días de estancia promedio por servicio

Pregunta que responde: ¿qué servicio tiene, en promedio, las hospitalizaciones más largas? (solo se cuentan las que ya tienen alta).

Relevancia: la estancia promedio es uno de los indicadores más usados para planear cuántas camas necesita cada servicio, y para detectar si un servicio se está demorando más de lo esperado en dar salida a sus pacientes.

Etapas: `$match` → `$project` → `$group` → `$project` → `$sort`.

```
db.hospitalizaciones.aggregate([
  { $match: { alta: { $ne: null } } },
  { $project: { servicio: 1, dias_estancia: { $dateDiff: {
        startDate: { $toDate: "$fecha_admision" },
        endDate: { $toDate: "$alta.fecha_alta" }, unit: "day" } } } },
  { $group: { _id: "$servicio", estancia_promedio_dias: { $avg: "$dias_estancia" },
      total_casos: { $sum: 1 } } },
  { $project: { _id: 0, servicio: "$_id",
      estancia_promedio_dias: { $round: ["$estancia_promedio_dias", 1] }, total_casos: 1 } },
  { $sort: { estancia_promedio_dias: -1 } }
])
```

Nota técnica real: la primera versión de esta consulta intentaba redondear con `$round` directamente dentro del `$avg`, en la misma etapa `$group`, y falló con `MongoServerError: The $round accumulator is a unary operator`. Se corrigió separando el redondeo en una etapa `$project` aparte, después de agrupar (ver `evidencia_ia.md`, caso 3).

Evidencia real de esta ejecución: ver Figuras 10 y 11, sección 5.2.

4.4 Consulta 4 — Diagnósticos más frecuentes

Pregunta que responde: ¿qué diagnósticos CIE-10 son los más comunes en la base, y en qué servicios aparecen?

Relevancia: este es justo el tipo de dato que alimenta la vigilancia epidemiológica departamental: si de repente sube mucho un diagnóstico respiratorio en varios hospitales a la vez, podría ser el inicio de un brote. Con los pocos datos de prueba de esta base cada diagnóstico sale con frecuencia 1; con datos reales de producción esta misma consulta sí mostraría patrones repetidos.

Etapas: `$unwind` → `$group` → `$sort` → `$project` → `$limit`.

```
db.hospitalizaciones.aggregate([
  { $unwind: "$diagnosticos" },
  { $group: { _id: { codigo: "$diagnosticos.codigo_cie10", descripcion: "$diagnosticos.descripcion" },
      frecuencia: { $sum: 1 }, servicios: { $addToSet: "$servicio" } } },
  { $sort: { frecuencia: -1 } },
  { $project: { _id: 0, codigo_cie10: "$_id.codigo", descripcion: "$_id.descripcion",
      frecuencia: 1, servicios: 1 } },
  { $limit: 10 }
])
```

Evidencia real de esta ejecución: ver Figuras 11 y 12, sección 5.2.

5. Evidencia de ejecución (capturas reales de mongosh)

Las siguientes 12 capturas son del scroll continuo de la terminal durante la ejecución real de los 5 scripts, en orden cronológico. Como son capturas de scroll continuo, varias se traslapan un poco en contenido con la siguiente (la cola de una repite en el inicio de la que sigue); se dejaron completas, sin recortar, para no alterar la evidencia original.

5.1 Evidencia de ejecución CRUD (scripts 01 a 04)

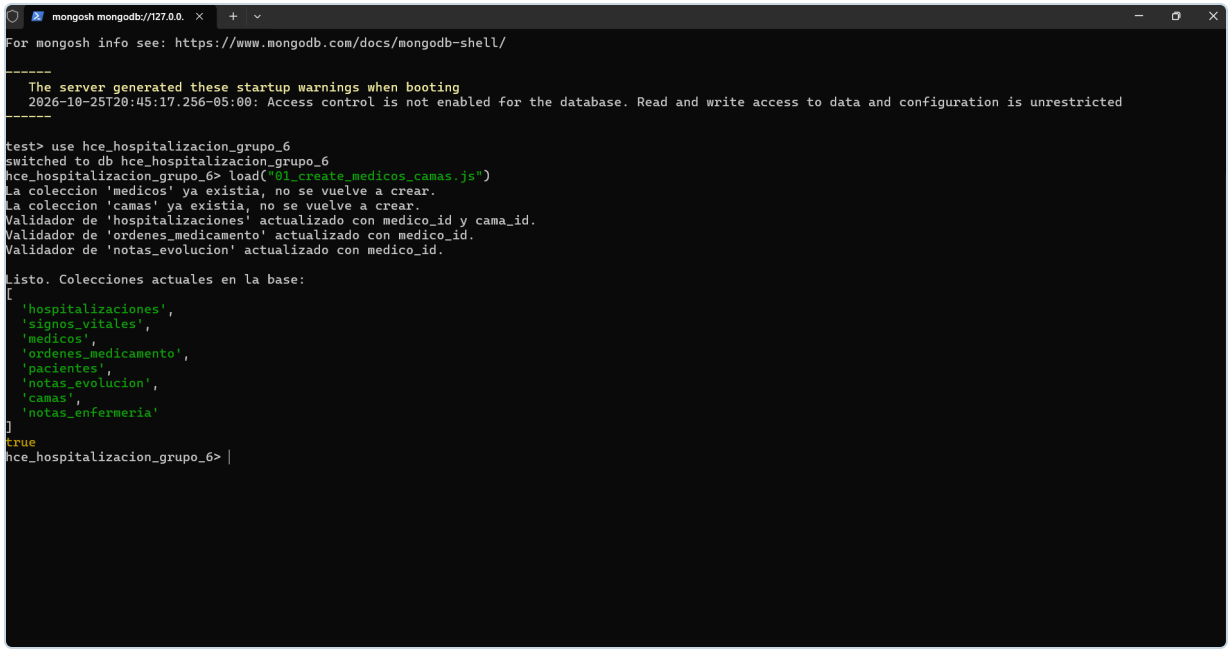


Figura 1. Salida completa de `01_create_medicos_camas.js`: validadores actualizados y lista final de colecciones, antes de correr el script 02.



Figura 2. Cola de la Figura 1 + salida completa de `02_insert_medicos_camas.js` ("Total medicos: 7, Total camas: 16").

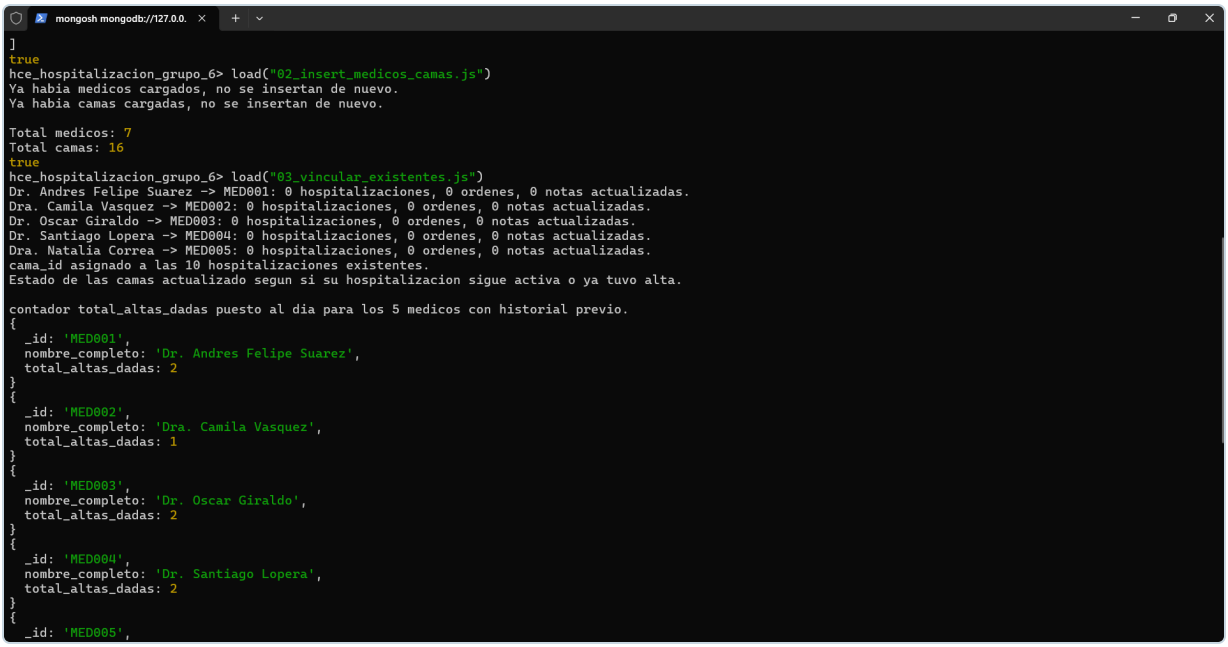


Figura 3. `03_vincular_existentes.js`: asignación de `medico_id`/`cama_id` y contador `total_altas_dadas` para MED001 a MED004.

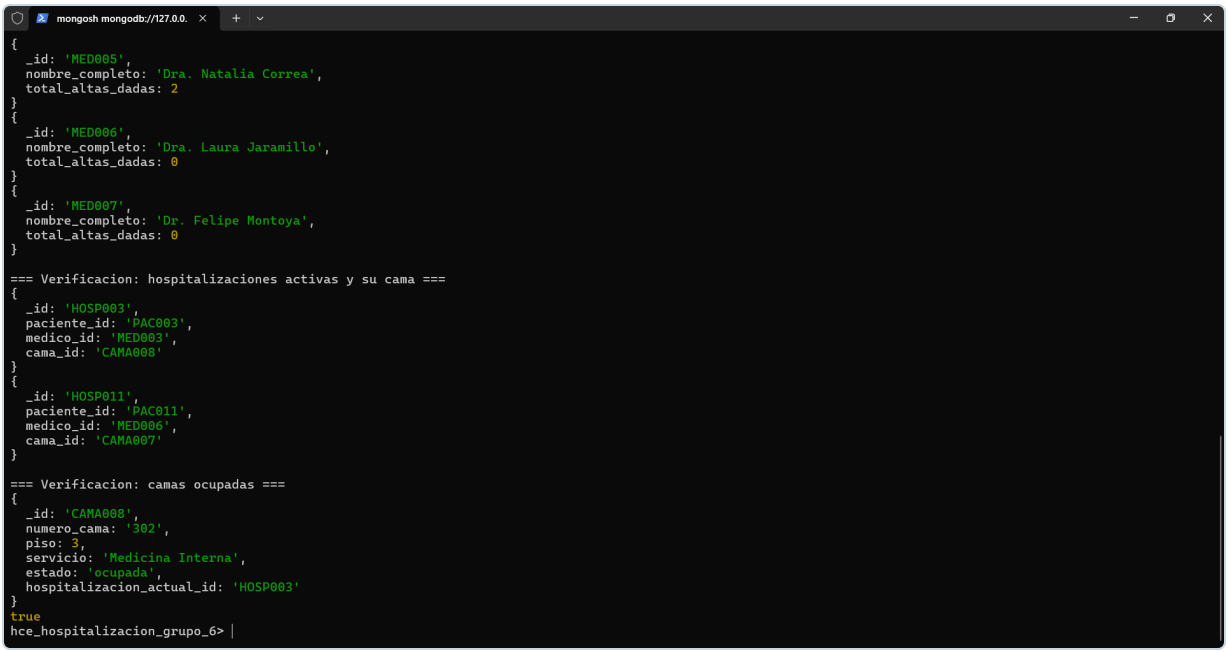


Figura 4. Continúa la Figura 3: MED005 a MED007, y las verificaciones de hospitalizaciones activas (HOSP003, HOSP011) y camas ocupadas (CAMA008).

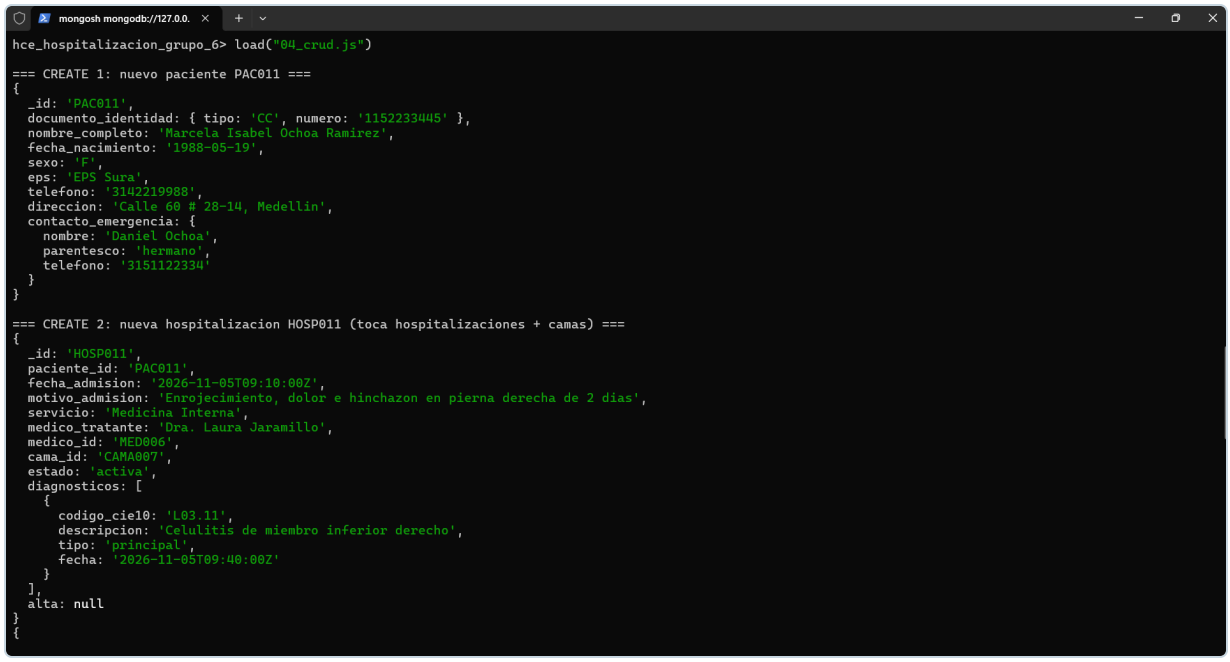


Figura 5. Inicio de `04_crud.js`: CREATE 1 (paciente PAC011 completo) y CREATE 2 (documento de la hospitalización HOSP011).

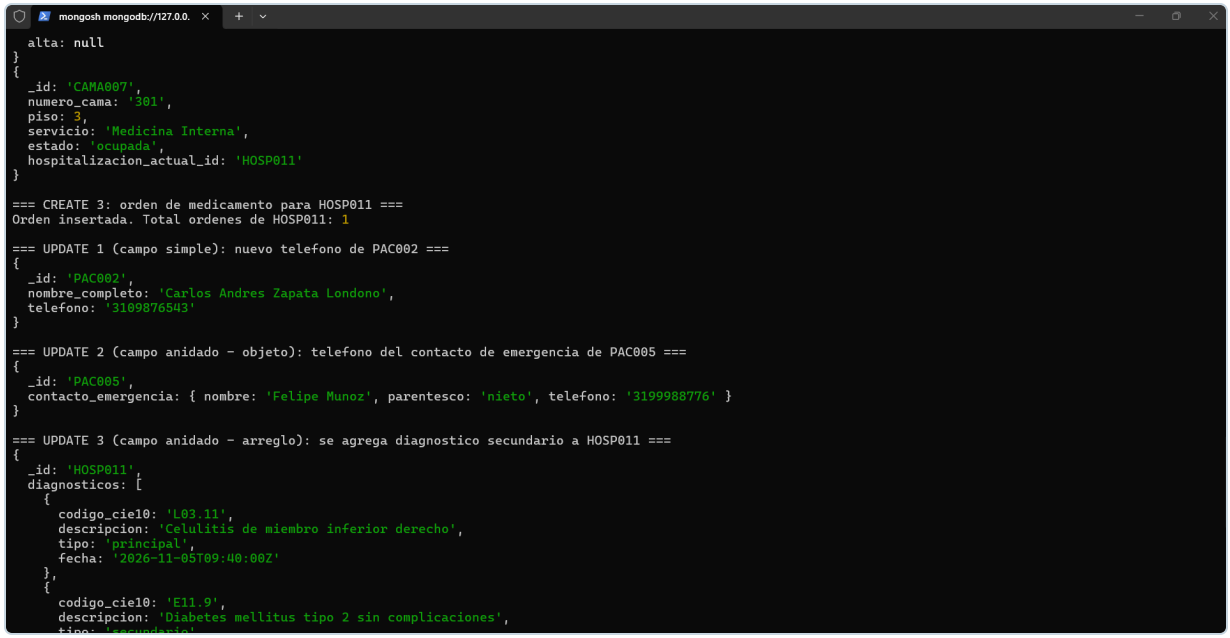


Figura 6. Continúa CREATE 2 (cama CAMA007 ocupada), CREATE 3 (orden de medicamento), UPDATE 1 (teléfono PAC002) y UPDATE 2 (contacto de emergencia PAC005).

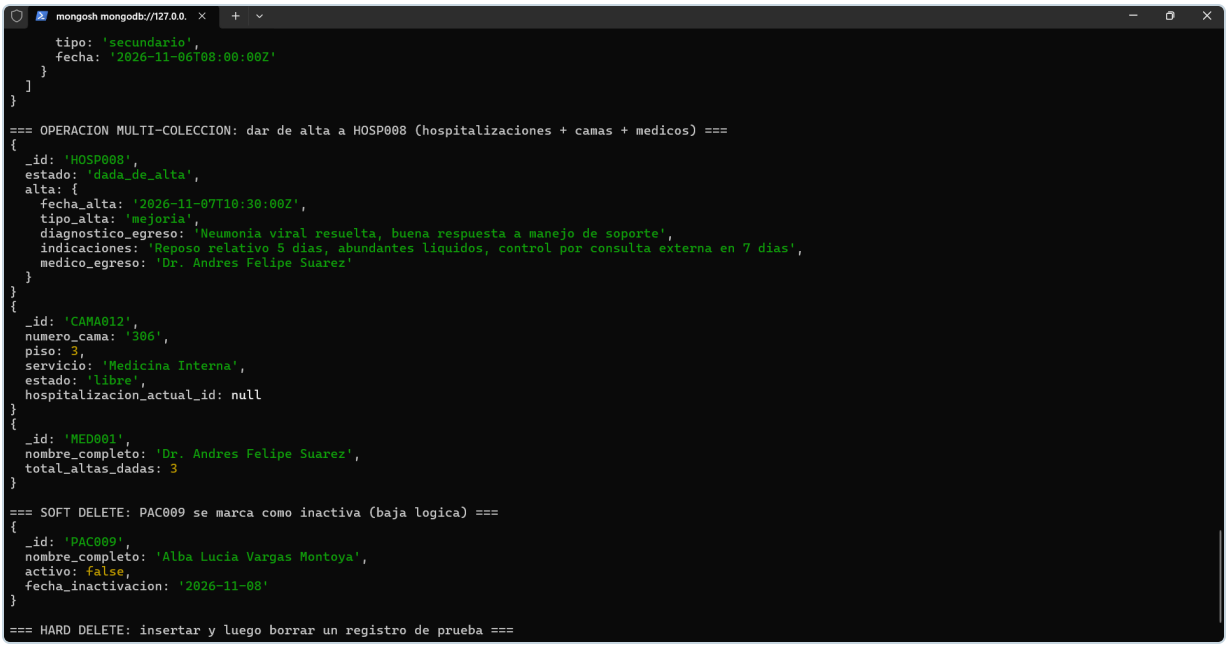


Figura 7. Termina UPDATE 3 (diagnóstico secundario agregado), operación MULTI-COLECCIÓN completa (alta de HOSP008, cama CAMA012 libre, MED001 con `total_altas_dadas: 3`) e inicio del SOFT DELETE (PAC009 inactiva).

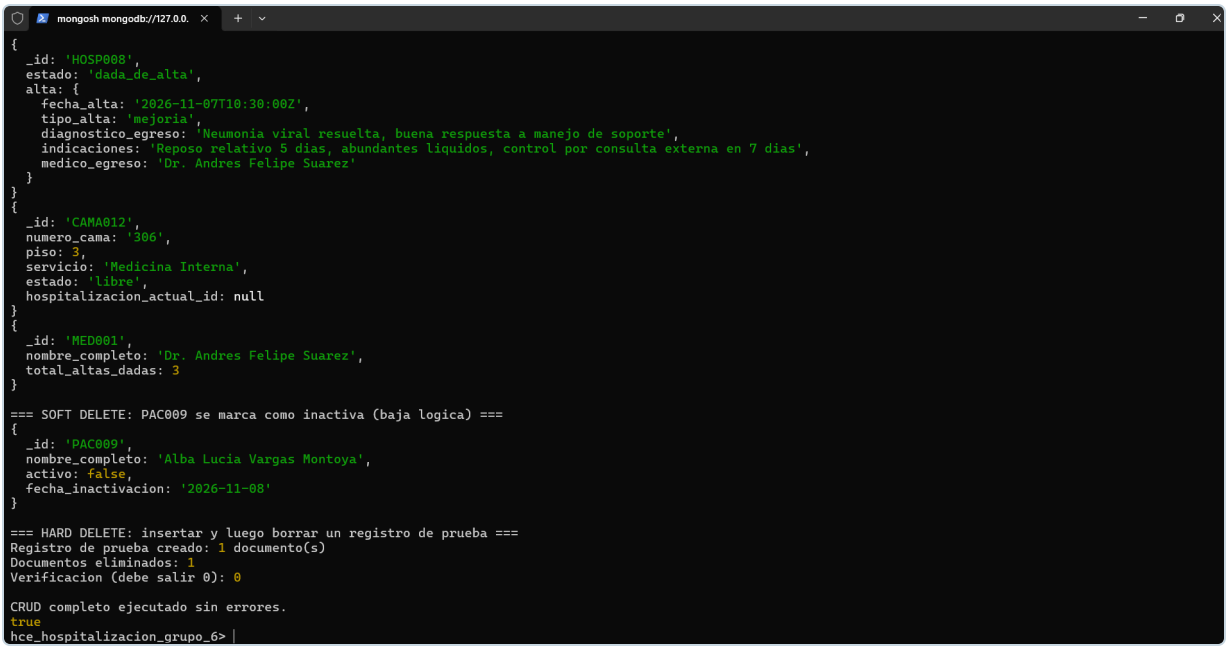


Figura 8. Repite la operación multi-colección/soft delete y termina el HARD DELETE (creado 1, eliminado 1, verificación 0) con el mensaje "CRUD completo ejecutado sin errores".

5.2 Evidencia de consultas avanzadas (script 05)

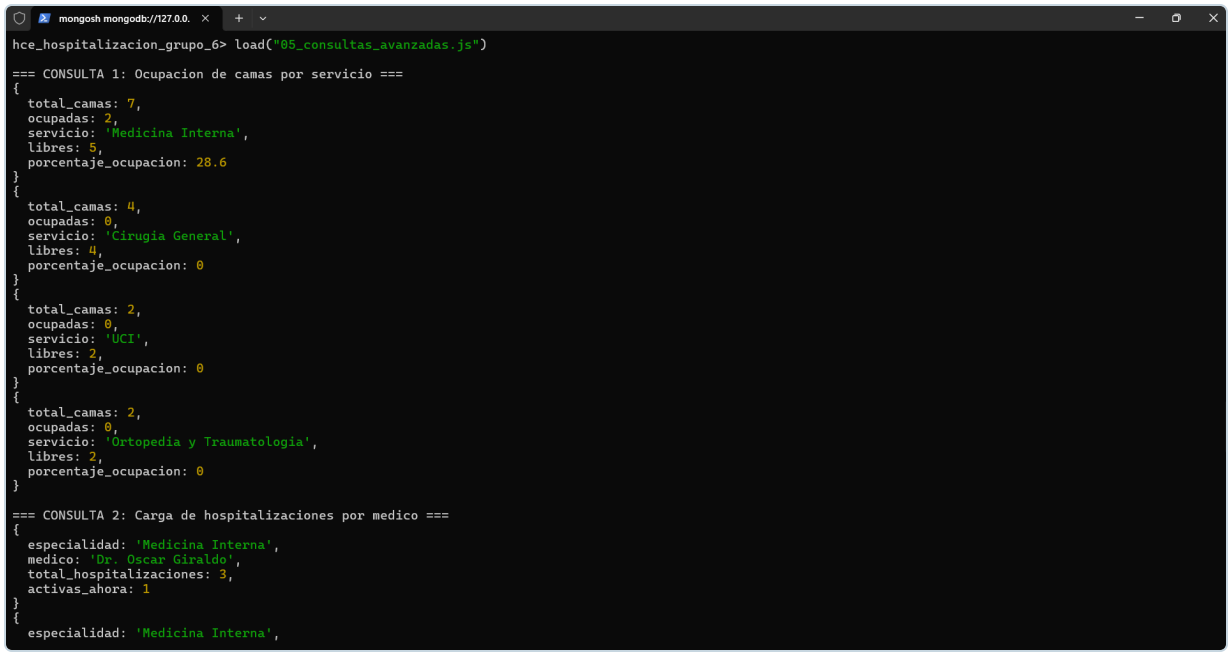


Figura 9. Inicio de `05_consultas_avanzadas.js`: Consulta 1 completa (ocupación de camas por servicio) e inicio de la Consulta 2 (Dr. Oscar Giraldo).

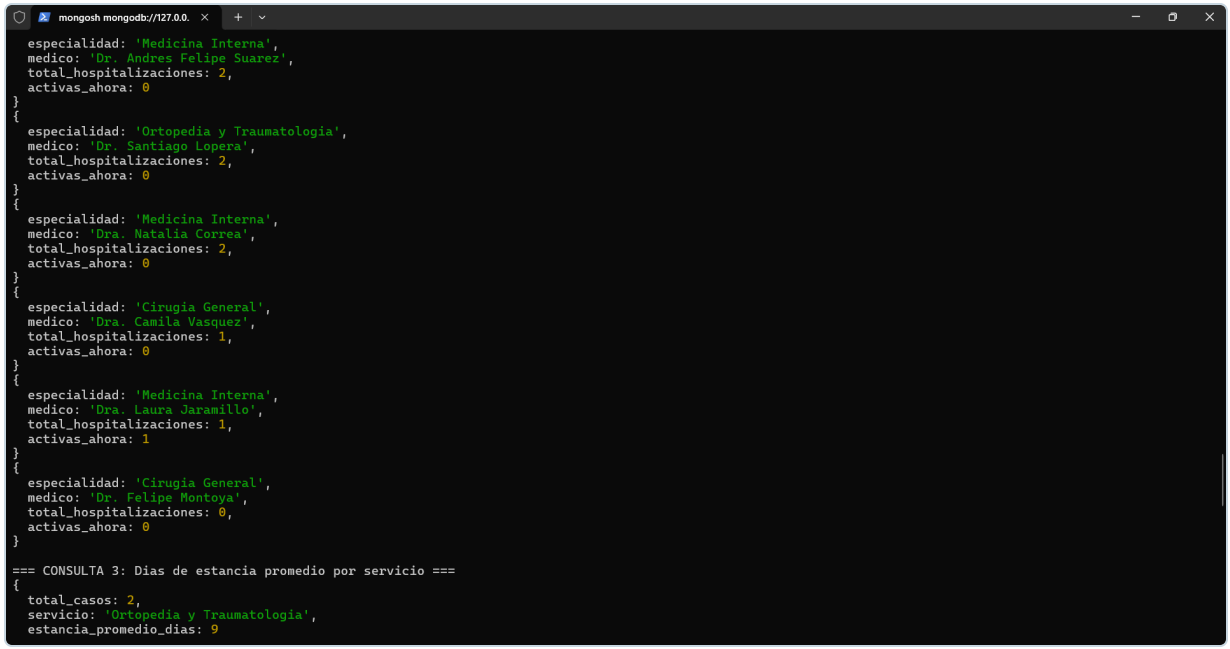


Figura 10. Continúa la Consulta 2 (resto de médicos, incluyendo los que tienen carga 0) e inicio de la Consulta 3 (Ortopedia y Traumatología, 9 días).

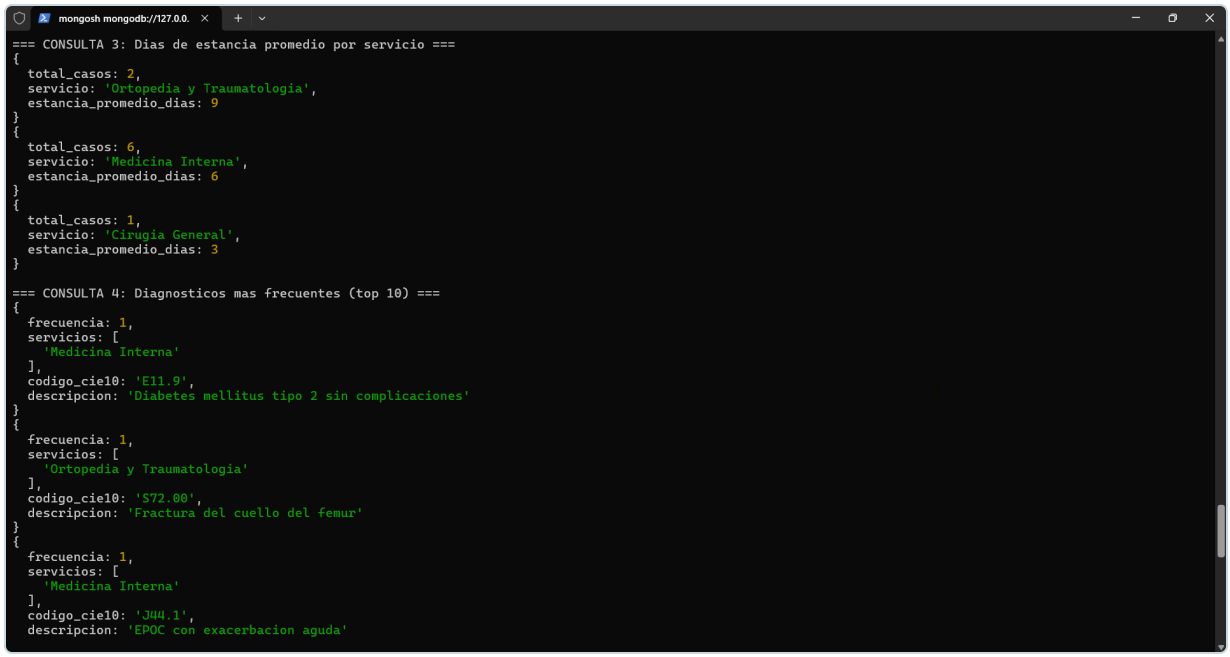


Figura 11. Termina la Consulta 3 (Medicina Interna 6 días, Cirugía General 3 días) e inicio de la Consulta 4 (primeros diagnósticos más frecuentes).

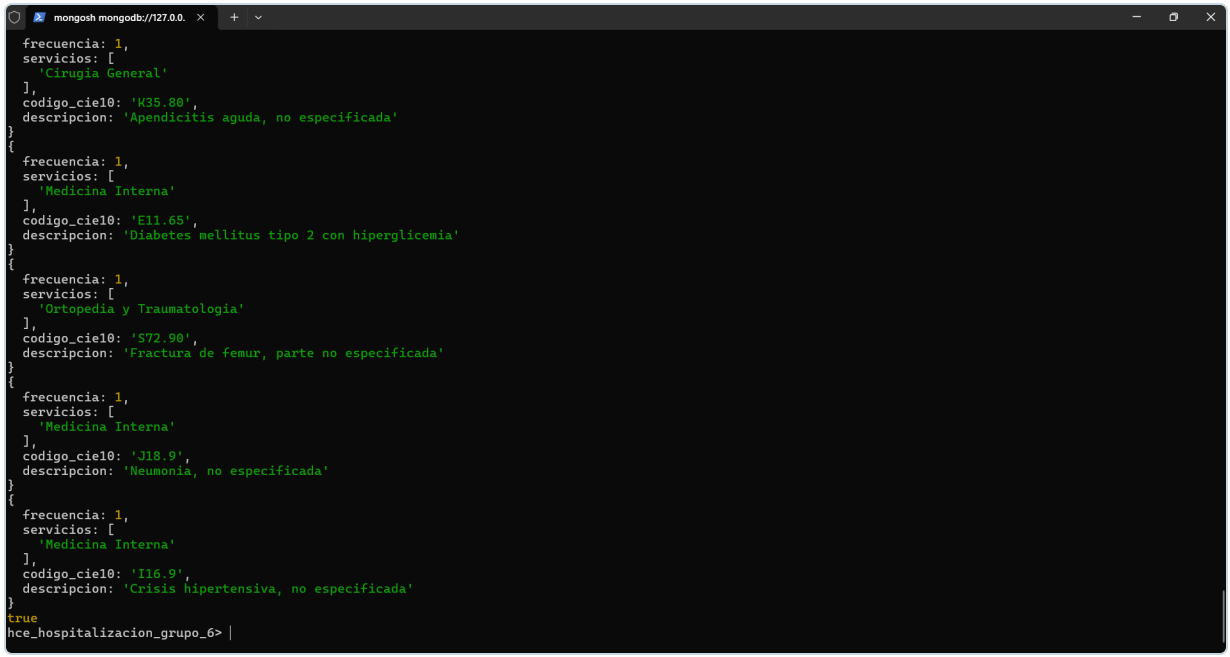


Figura 12. Termina la Consulta 4: resto de diagnósticos más frecuentes (K35.80, E11.65, S72.90, J18.9, I16.9).

6. Evidencia de uso de IA

(Contenido completo y trazable en `evidencia_ia.md`.) **Herramienta:** Claude (Anthropic), modelo Claude Sonnet 5, a través de Claude Code.

> **Caso 1 — Diseño del ajuste del esquema.** Prompt: diseñar cómo agregar médicos y camas justificando embedding vs. referencing. Resultado: `schema.json` con la estrategia y la denormalización controlada. Qué mejoró: evitó repetir el error de dejar al médico como texto suelto, que no permite medir carga de trabajo ni disponibilidad en tiempo real.

> **Caso 2 — Generación de los scripts ejecutables.** Prompt: generar los 5 scripts con comentarios en tono de estudiante. Resultado: `01` a `05` de esta carpeta. Qué mejoró: pasó el diseño de papel a una base real poblada en minutos.

> **Caso 3 — Depuración de un error real.** Al ejecutar la Consulta 3, Mongo devolvió `MongoServerError: The $round accumulator is a unary operator`. La IA identificó que `$round` no puede envolver a `$avg` en la misma etapa `$group` en esta versión, y separó el redondeo en un `$project` aparte. Qué mejoró: corrigió un bug real verificado contra el motor, no uno hipotético.

> **Caso 4 — Autoevaluación contra la rúbrica real del curso.** Prompt: autoevaluar el entregable contra los 5 criterios de la rúbrica y mejorar lo que hiciera falta, sin inflar lo que no se puede controlar (video, GitHub). Resultado: el contador `total_altas_dadas`, la operación multi-colección de dar de alta, los comentarios de "por qué" en el CRUD, la relevancia clínica de cada consulta, y este formato trazable de evidencia de IA. Qué mejoró: forzó a dejar explícito el razonamiento detrás de cada decisión, no solo el código que funciona.

[CAPTURA PENDIENTE: fragmento de la conversación con Claude mostrando el diseño de medicos/camas y la corrección del error de $round — esta es una captura distinta a las 12 de mongosh, todavía no se ha tomado]

7. Reflexión ética

(Texto completo en `reflexion_etica.md`.) La reflexión aborda explícitamente cuatro ángulos distintos, cada uno con un ejemplo concreto de este mismo modelo:

- **Privacidad:** cruzar médico + diagnóstico (Consulta 2) también expone datos sensibles del personal de salud, no solo del paciente.
- **Trazabilidad:** el contador `medicos.total_altas_dadas` no deja rastro automático de cómo cambió en el tiempo ni quién lo cambió — a diferencia de un trigger o tabla de auditoría en el modelo relacional.
- **Gobernanza:** esta instalación local no tiene todavía roles ni control de acceso configurado; en producción habría que restringir, por ejemplo, que enfermería no pueda escribir en `medicos` ni en diagnósticos médicos.
- **Sesgos:** las notas en texto libre (`notas_evolucion`, `notas_enfermeria`) permiten registrar el mismo caso de formas muy distintas según quién lo documente, y la Consulta 4 (diagnósticos frecuentes) solo puede reflejar a los pacientes que lograron ser admitidos, invisibilizando a quienes no tuvieron acceso.

8. Conclusiones

- El modelo documental de la Unidad 2 se pudo ampliar sin romper nada de lo ya entregado, gracias a que MongoDB no exige declarar todos los campos por adelantado — aunque esa misma característica es, a la vez, el riesgo ético discutido en la sección 7.
- Referenciar `medicos` y `camas` (en vez de embeberlos) fue la decisión correcta porque ambas entidades se reutilizan constantemente desde múltiples documentos.
- La operación de "dar de alta" mostró en la práctica el reto más importante de trabajar sin transacciones multi-documento: mantener 3 colecciones relacionadas coherentes entre sí depende de la disciplina de la aplicación, no del motor.
- Las 4 consultas avanzadas muestran que, con pocas colecciones adicionales, se pueden responder preguntas operativas reales de un hospital: ocupación de camas, carga médica, tiempos de estancia y patrones de diagnóstico.

Video explicativo

Guion y checklist completo en `guion_video.md` (incluye explícitamente mostrar la operación multi-colección y las 4 consultas). Enlace del video: *[PENDIENTE — pegar aquí el enlace una vez grabado y subido]*.

Repositorio digital

[PENDIENTE — pegar aquí el enlace del repositorio de GitHub una vez subido].

Informe generado con apoyo de IA (Claude, Anthropic) — ver sección 6, Evidencia de uso de IA.